RDB2RDF Views

No Author Given

No Institute Given

1 Basic Concepts and Notation

As usual, a *relation scheme* is denoted as $R[A_1, \ldots, A_n]$. The *relational constraints* considered in this article consist of *mandatory* (or not null) *attributes, keys, primary keys* and *foreign keys*. In particular, $F(R{:}L, S{:}K)$ denotes a foreign key, named F, that *relate* R and S, where L and K are lists of attributes from R and S, respectively, with the same length.

A *relational schema* is a pair $\boldsymbol{S} = (\boldsymbol{R}, \Omega)$, where $\boldsymbol{R}$ is a set of relation schemes and Ω is a set of relational constraints such that: (i) Ω has a unique primary key for each relation scheme in $\boldsymbol{R}$; (ii) Ω has a mandatory attribute constraint for each attribute which is part of a key or primary key; (iii) if Ω has a foreign key of the form $F(R{:}L, S{:}K)$, then Ω also has a constraint indicating that K is the primary key of S. The *vocabulary* of $\boldsymbol{S}$ is the set of relation names, attribute names, and foreign key names used in $\boldsymbol{S}$. Given a relation scheme $R[A_1, \ldots, A_n]$ and a *tuple variable* t over R, $t.A_k$ denotes the *projection* of t over A_k. *Selections* over relation schemes are defined as usual.

A *state* σ of a relational schema $\boldsymbol{S}$ assigns to each relation scheme R of $\boldsymbol{S}$ a relation $R(\sigma)$, in the usual way. Let $\boldsymbol{S} = (\boldsymbol{R},\Omega)$ be a relational schema and R and T be relation schemes of $\boldsymbol{S}$.

An *ontology vocabulary*, or simply a *vocabulary*, is a set of *class names, object property names* and *datatype property names*. An *ontology* is a pair $\boldsymbol{O} = (V,\Sigma)$ such that V is a vocabulary and Σ is a finite set of formulae in V, the *constraints* of $\boldsymbol{O}$. The constraints include the definition of the *domain* and *range* of a property, as well as *cardinality constraints*, defined in the usual way.

A *foreign key* is an expression of the form $F(R : K, S : L)$, where:

- F is the *name* of the foreign key;
- R and S are relation schemes;
- K and L are lists of attributes of R and S, respectively;
- $|K| = |L|$.

The relation schemes R and S are said to be *related* by F. The foreign key $F(R : K, S : L)$ induces a binary relationship between tuples of R and S and may be traversed in either direction, from R to S or from S to R. Lastly, a tuple $r \in R(\sigma)$ is related (or connected) to a tuple $s \in S(\sigma)$ with respect to F if and only if the values of the attributes in K of r match the values of the attributes in L of s.

A *relational path* φ is a list of the form $[e_1, \ldots, e_{n-1}]$, where

- e_j is either a foreign key name F_j, indicating that the path traverses F_j from R_j to R_{j+1}, or an expression of the form $(F_j)^{-1}$, indicating that the path traverses the foreign key F_j from R_{j+1} to R_j, in the reverse direction;
- when e_j is traversed in the forward direction, then the second relation in e_j must be equal to the first relation of e_{j+1};
- when e_j is traversed in the reverse direction, then the first relation in e_j must be equal to the first relation of e_{j+1}.

The set of all relation schemes occurring along φ is denoted by $Relations(\varphi)$.

A *prefix* of φ is any sequence $\phi = [e_1, \ldots, e_k]$ with $0 \leq k \leq n-1$.

Given a database state σ, a tuple $r_1 \in R_1(\sigma)$ *is connected to* (or *references*) a tuple $r_n \in R_n(\sigma)$ *by following the path* $\varphi = [e_1, \ldots, e_{n-1}]$ if and only if there exists a sequence of tuples $r_2, \ldots, r_{n-1}$ with $r_i \in R_i(\sigma)$ for $2 \leq i \leq n-1$ such that, for each $i \in \{1, \ldots, n-1\}$, the tuples r_i and r_{i+1} are related with respect to the foreign key F_i in the forward or reverse direction.

A relational path φ induces a *predicate path* $\varphi(x, y)$, which is evaluated in a database state by joining the relations in the path in the directions indicated, using the lists of attributes in the foreign key.

An example of a predicate path with two foreign keys is

$$[(place_annotation_fk_place)^{-1},\ place_annotation_fk_annotation](p, an)$$

Note that the first expression $(place_annotation_fk_place)^{-1}$ indicates that the foreign key is traversed in the reverse direction.

Table 1: Transformation Rule Patterns

TR	**Transformation Rule Pattern**
CTR	$\psi : C(s) \leftarrow R(r), URI_R(r, s), \delta[r]$ where - ψ is the name of the transformation rule - C is a class in the vocabulary V - R is the name of a relation or a view in the source schema $\boldsymbol{S}$ - r is a tuple variable bound to tuples of R - s is the URI associated with the tuple bound to r - $URI_R(r, s)$ is a URI constructor predicate as defined in the text - $\delta[r]$ is an optional selection over R, among other relations
Local DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \ldots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, \text{“}A_1'', \text{“}G'', v_1), \ldots, RDFLiteral(r.A_k, \text{“}A_k'', \text{“}G'', v_k), T([v_1, ..., v_k], v), \delta[r]$ where - ψ is the name of the transformation rule - P is a datatype property of V with domain D - $nonNull$ and $RDFLiteral$ are predicates as defined in the text - “$R_D(r), B_D[r, s]$” is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - $A_1, \ldots, A_k$ are attributes of R_D - T is an optional function that transforms values of attributes $A_1, \ldots, A_k$ of R_D to values of property P - $\delta[r]$ is an optional selection over R_D, among other relations
Path DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \ldots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, \text{“}A_1'', \text{“}G'', v_1), \ldots, RDFLiteral(r.A_k, \text{“}A_k'', \text{“}G'', v_k), T([v_1, ..., v_k], v),$ $\varphi(r, g), \delta[r, g]$ where - $\varphi(r, g)$ is a predicate path starting on R_D and ending on a relation R_G - $\delta[d, g]$ is an optional selection over R_D and R_G, among other relations - the other terms are as in a Path DTR
Regular OTR	$\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], R_G(g), B_G[g, o], \varphi(d, g), \delta[d, g]$ where - “$R_G(g), B_G[g, o]$” is the RHS of the CTR ψ_G that matches class G with pivot relation R_G of ψ_G - the other terms are as in a Path DTR
Reification OTR	$\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], nonNull(r.A_1), \ldots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, \text{“}A_1'', \text{“}G'', v_1), \ldots, RDFLiteral(r.A_k, \text{“}A_k'', \text{“}G'', v_k), T([v_1, ..., v_k], v),$ $RDFIRI(u, o), \delta[r]$ where - $RDFIRI(u, o)$ is as defined in the text - the other terms are as in a Local DTR

2 Specification of Entity-Preserving RDB2RDF View

This section presents the formalism for specifying entity-preserving RDB2RDF views. By restricting ourselves to this class of views, we can precisely identify the tuples relevant to a data source update w.r.t. an RDB2RDF view.

The formal definition of an RDB2RDF view is similar to that given in [?,?]. An *RDB2RDF view* is a triple $\mathcal{W} = (V, \boldsymbol{S}, M)$, where:

1. V is the vocabulary of the *target ontology*;
2. $\boldsymbol{S}$ is the *source relational schema*; and
3. M is a set of *mappings* between V and $\boldsymbol{S}$, defined by *transformation rules* (TRs).

A view $\mathcal{W} = (V, \boldsymbol{S}, M)$ satisfies the *entity-preserving property* iff:

- the instances of the classes in V correspond to tuples in selected relations of $\boldsymbol{S}$, which are called the *pivot relations* of $\mathcal{W}$, so that distinct tuples from the same pivot relation generate distinct class instances;
- the values of datatype properties in V of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in V correspond to relationships between tuples of the pivot relations of $\boldsymbol{S}$.

Intuitively, a view satisfies the entity-preserving property iff it preserves the base entities (objects) of the source database, rather than creating new entities from the existing ones [?].

The problem of generating transformation rules is addressed in [?,?] and is outside of the scope of this work. However, Table 1 summarizes a set of Transformation Rule (TR) patterns that lead to the definition of relational to RDF mappings that guarantee that the RDB2RDF views satisfy the entity-preserving property by construction.

Table 1 defines three types of TR patterns: *Class Transformation Rule* (CTR), *Datatype Property Transformation Rule* (DTR), and *Object Property Transformation Rule* (OTR).

Intuitively, a CTR ψ maps tuples of a relation R into RDF instances of a class C. It establishes a semantic correspondence between a pivot tuple r and an RDF resource x, such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDB2RDF view.

There are two types of DTRs. A *Local DTR* has no relational path in the body so that the datatype values are obtained directly from attribute values of the pivot tuples. A *Path DTR* has a relational path in the body, and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

There are also two types of OTRs. A *regular OTR* ψ defines object properties by relating an RDF instance generated from a pivot tuple t to an RDF instance generated from a pivot tuple u reached from t through a relational path in the source schema. Hence, OTRs capture relationships between entities represented in the database.

A *reification OTR* relates an RDF instance generated from a pivot tuple t to a URI obtained by reification of a literal l created from attribute values of a pivot tuple u reached from t through a relational path in the source schema.

In addition to predicate paths, the transformation rules in Table 1 and in the experiments use several predicates. The simplest ones are:

- *nonNull(v)* holds iff value v is not null.
- *RDFLiteral(u,A,R,v)* holds iff v is the literal representation of a value u of attribute of A of R.
- *RDFIRI(u,o)* holds iff the non-null value u is converted into the RDF IRI o (that is, o is just the literal u treated as a URI).
- *F(r,s)* holds iff a tuple r of R is related to tuple s of S by a foreign key of the form *F(R:L,S:K)*.

The more complex predicates are as follows.

Let R be the pivot relation of a transformation rule ψ, and r be a pivot tuple of R.

A *URI template* for R is a string pattern p with place holders $\langle h_1, ..., h_n \rangle$ such that, when the place holders are appropriately instantiated, the resulting string becomes a URI.

A URI s *is associated with* r *via* p iff the predicate

$$hasURI(p, \langle e_1, \ldots, e_n \rangle, s)$$

holds, where

- $\langle e_1, \ldots, e_n \rangle$ is an ordered sequence of expressions, where each e_i is of the form:
  - $e_i = r.A$, where $r.A$ denotes an attribute of the pivot tuple.

- $e_i = [FK(r,t)/t.A]$, where $[FK(r,t)/t.A]$ denotes the value of attribute A obtained after following the foreign key FK from the pivot tuple r to tuple t such that the foreign-key navigation must be functional from the pivot relation to the referenced relation.

– s is the result of instantiating each h_i by e_i in p.

To simplify the notation, a second predicate, called a *URI constructor*, is introduced by definition:

$$URI_R(r,s) \equiv hasURI(p, \langle e_1, \ldots, e_n \rangle, s)$$

As an example, the first line of Table 1 defines the URIs associated with the pivot relation *artist* using the predicate

$$hasURI(\text{“http://musicbrainz.org/artist/}\langle h \rangle\text{”}, \langle r.gid \rangle, s)$$

indicating that s is generated by replacing h by “*r.gid*” in the template

$$\text{“http://musicbrainz.org/artist/}\langle h \rangle\text{”}$$

If the transformation rule is generated from an R2RML statement, the R2RML subject-map template specifies the complete URI template, and $\langle e1, \ldots, en \rangle$ is the ordered sequence of expressions corresponding to the placeholders of the R2RML URI template. Furthermore, every placeholder for an rr:template is translated into exactly one pivot-rooted attribute expression, preserving the complete URI template, the placeholder order, and the relational origin of each value.

The approach proposed in this article efficiently computes changesets by exploring the entity-preserving property, which allows the precise identification of the tuples in the pivot relations whose corresponding instances may have been affected by an update. The advantages of this approach are: (1) it simplifies the specification of the view, as the formalism is specifically designed to describe the correspondences that define an entity-preserving view; (2) the restrictions on the structure of the queries help ensure the consistency of the specification of the view; (3) the formal expressions that define the queries can be explored to construct the procedures automatically that compute the changesets that maintain the RDB2RDF view; (4) it facilitates the task of providing rigorous proofs for the correctness of the proposed approach.

3 Materialization of an RDB2RDF View

The state of an RDB2RDF view is represented as a set of quadruples (or *quads*) of the form (s,p,o,g). In standard N-Quads, the fourth element g identifies the named graph to which the triple (s,p,o) belongs. In this approach, each transformation rule ψ is associated with a named graph, and its URI is used as the fourth element. Thus, each quad (s,p,o,g), where g is the URI of a transformation rule ψ, indicates that the triple (s,p,o) was generated by ψ.

This representation allows the RDB2RDF view to be materialized as a set of quads partitioned by transformation rules. Intuitively, each rule ψ induces a named graph containing exactly the triples generated by ψ, which provides a simple and operational mechanism for provenance tracking and avoids ambiguity in the presence of duplicate triples generated by different rules.

The state of an RDB2RDF View is defined compositionally in three hierarchical levels.

Definition 1 (Hierarchical RDF State Semantics). *Let $W = (V,S,M)$ be an RDB2RDF view and let σ be a relational database state.*

1. ***RDF Contribution produced by a Transformation Rule***
   Let $\psi \in M$ be a transformation rule whose pivot relation is R^*, with tuple variables $r_1, \ldots, r_n$ associated with relation schemes $R_1, \ldots, R_n$.
   For tuples $p_1 \in R_1(\sigma), \ldots, p_n \in R_n(\sigma)$, the application of ψ under σ, denoted by

   $$\psi[r_1 \leftarrow p_1, \ldots, r_n \leftarrow p_n](\sigma)$$

   is defined as the set of RDF triples produced by evaluating ψ with bindings $r_i = p_i$, for $i = 1, \ldots, n$, over σ.
2. ***RDF Contribution of a Pivot Tuple under a Rule.*** *For a pivot tuple $t \in R^*(\sigma)$, the RDF contribution of t under ψ and σ, denoted by $RDF_State[\psi](t,\sigma)$, is defined as:*

   $$RDF_State[\psi](t,\sigma) = \{(s,p,o,g) \mid (s,p,o) \in \psi[r \leftarrow t](\sigma),\ g = URI(\psi)\}$$

3. ***RDF State of the View W.***
The RDF state of the view W under σ, denoted by $RDF_State(W, \sigma)$, is defined as:

$$RDF_State(W, \sigma) = \bigcup_{\psi \in M} \bigcup_{t \in pivot(\psi)(\sigma)} RDF_State[\psi](t, \sigma)$$

Intuitively, the RDF state of an RDB2RDF view is obtained by aggregating the RDF contributions generated by all transformation rules over all pivot tuples in the database state.

The RDF state can be naturally interpreted as an RDF dataset, where each quad (s, p, o, g) represents a triple (s, p, o) in the named graph identified by $g = URI(\psi)$. Named graphs are used to encode rule-level provenance.